

ReBeL, in this codebase

from the idea to the code — a guided tour of `trainer/rebel_py`

Combining Deep RL and Search on public belief states, implemented for heads-up no-limit hold'em

Companion to `docs/modern_poker_ai_tutorial.pdf` — that document is the *theory*; this one is the *map from theory to code*.

Intuition

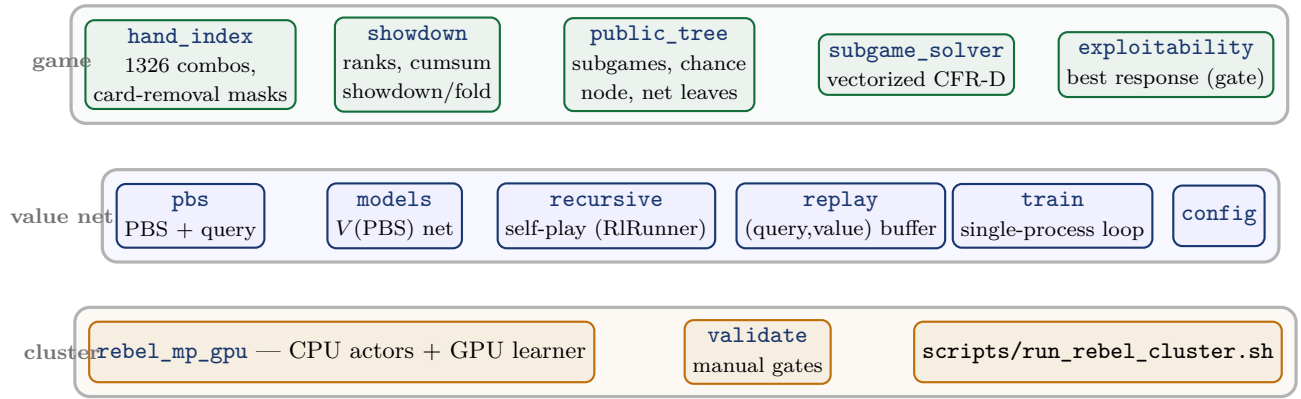
ReBeL's one idea (tutorial, §ReBeL): an imperfect-information game becomes a *perfect-information* game if the state is the **public belief state** (PBS). Then AlphaZero-style self-play + search applies, with **CFR** doing the searching and a **value net** $V(\mathbf{PBS})$ pricing the leaves of depth-limited subgames. This codebase is a paper-faithful, from-scratch implementation of that idea over the project's poker engine. Every ReBeL concept below is named with the file and function that realizes it.

Contents

1	High-level view: the module map	1
2	The conversion trick in code: PBS and the query (<code>pbs.py</code>)	2
3	The depth-limited subgame (<code>public_tree.py</code>)	2
4	The search engine: vectorized CFR-D (<code>subgame_solver.py</code>)	3
5	The value network (<code>models.py</code>)	4
6	The self-play loop (<code>recursive.py</code>)	5
7	Training and the gate (<code>train.py</code> , <code>replay.py</code> , <code>exploitability.py</code>)	6
8	Scaling the chance node (<code>showdown.py</code>)	6
9	The cluster trainer (<code>rebel_mp_gpu.py</code>)	7
10	Correspondence at a glance, and what is validated	7

1 High-level view: the module map

The package `trainer/rebel_py/` is pure Python, vectorized over the **1326** hole-card combinations with NumPy, and reuses the project's pybind engine (`pokertrainer_engine`) only to enumerate legal bets and evaluate 7-card hands. It is built in layers: ground-truth game machinery at the bottom, the value-net self-play loop in the middle, the cluster trainer on top. The layers were validated bottom-up (each has a hard correctness gate before the one above it was written).



The phases (how it was built and validated).

Phase	What	Gate (proof it is correct)	Commit
1	CFR-D solver, <i>no</i> net	push/fold = Nash oracle; river $\rightarrow$ 0 exploit.	7b3d718
2a	value net, river depth limit	self-play drops exploit. 2.0 $\rightarrow$ 0.04 bb/hand	a353fc7
2b	river chance node (turn $\rightarrow$ river)	ground truth $\rightarrow$ 0.027; equity = <code>evaluate7</code>	030b282, afe6928
3	cluster trainer (actors + learner)	river 1.1 $\rightarrow$ 0.9; checkpoints, TorchScript	8425c52
–	scaling (rank/incidence leaves)	exact vs dense; 1.7 GB $\rightarrow$ 5 MB, 110 $\rightarrow$ 11 s	83e1a75, 9e7217b

2 The conversion trick in code: PBS and the query (`pbs.py`)

Public belief state. Everything public (board, street, pot, who acts, amount to call, stack) *plus* both players' belief vectors over the 1326 hole combos. In code the public part is `PublicState` (a dataclass); the beliefs are two `np.ndarray` of length 1326 carried alongside.

The value net cannot eat a Python object; `encode_query` flattens a PBS (as seen by a chosen *traverser*) into the fixed float vector the net consumes. It mirrors `write_query_to` in the reference Liar's Dice C++ code, but with poker's richer public state:

Listing 1: `pbs.py` — the PBS $\rightarrow$ vector encoding. Two faithfulness points: beliefs are normalized to sum 1 (so the net learns value *per unit opponent mass*), with the same 10^{-80} smoothing so an all-zero reach maps to uniform, not NaN.

```
def encode_query(traverser, ps, reach0, reach1):
    q = np.empty(query_dim())          # 2710 floats
    q[0] = ps.to_act                    # who acts (0=SB / 1=BB)
    q[1] = traverser                    # whose value the net should output
    q[2] = ps.street / 3.0               # 0..3 (one net spans streets)
    q[3] = ps.pot_bb / POT_SCALE         # scalars normalized into [0,1]-ish
    q[4] = ps.to_call_bb / POT_SCALE
    q[5] = ps.stack_bb / POT_SCALE
    q[6:6+52] = board_multi_hot(ps.board) # which cards are out
    q[58:58+1326] = normalize_safe(reach0) # player-0 beliefs
    q[58+1326:] = normalize_safe(reach1)   # player-1 beliefs
    return q                             # dim = 6 + 52 + 2*1326
```

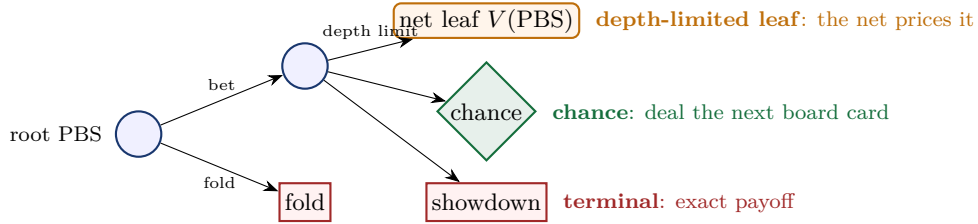
slice	meaning	width
q[0:6]	to_act, traverser, street, pot, to_call, stack	6
q[6:58]	board multi-hot (52 cards)	52
q[58:1384]	player-0 belief (normalized)	1326
q[1384:2710]	player-1 belief (normalized)	1326
total query_dim()		2710

Intuition

Why feed *both* players' beliefs and a `traverser` flag, rather than “my beliefs / your beliefs”? Because the subgame's value depends on the whole PBS, and ReBeL computes *all* beliefs (tutorial, Q “how does ReBeL know the opponent's beliefs?”): it never observes them, it *computes* them by Bayes from common-knowledge strategies. The `traverser` bit just selects which player's per-hand values the net should return for this query.

3 The depth-limited subgame (`public_tree.py`)

Tutorial step (2) is “build the depth-limited subgame — shape only, no leaf values yet”. That is exactly `build_river_subgame` / `build_turn_subgame`: they walk the engine to a public state, then enumerate the betting tree by cloning and stepping `Env`. A `Node` is a *public* state (betting sequence) — hole-independent, so one tree serves all 1326 hands. Leaves are of three kinds:



Terminal value closures. Each leaf stores a closure `term_value(traverser, opp_reach) → (1326,)` that returns the per-hand counterfactual value against the opponent's *reach* (so card removal is folded into the reach). Showdown and fold are derived in §8 without any 1326^2 matrix.

Net leaves are how the depth limit is realized: a non-terminal node at the limit becomes `is_net_leaf=True` and carries its `PublicState`, so the solver can build the query and ask the net. The depth limit follows the paper: *stop where new cards would be dealt*.

The chance node (Phase 2b) is the river card. The engine pre-deals all five board cards, so at the turn the active board is the first four and the river ranges over the 48 unseen cards. The betting *shape* is board-independent, so one river subtree is re-parametrized per card by its showdown. Card removal is handled exactly (next section).

Listing 2: `public_tree.py` — re-rooting/truncating an already-built tree. This is how the ReBeL recursion “builds a fresh subgame at each public state” without touching the engine: `subtree_subgame` copies the subtree from any node and turns chance children (or depth-limit nodes) into value-net leaves.

```
def subtree_subgame(full, root_nid, max_depth, stop_at_chance=False):
    # decision nodes at max_depth -> net leaves (carry their PublicState);
    # with stop_at_chance, a chance node's children become river-PBS net leaves
    # -> this is the ReBeL "turn subgame": solve turn betting, net values the river.
    ...
```

4 The search engine: vectorized CFR-D (`subgame_solver.py`)

Tutorial step (3): “ T_{cfr} iterations of tabular CFR inside G_t , with leaf evaluation *inside every iteration* — strategy and leaf values co-evolve.” `CfrSolver` is a faithful, vectorized port of the reference CFR in `subgame_solving.cc`: every per-node quantity is a $(1326, \text{n_actions})$ or $(1326,)$ array, so all 1326 hands are updated at once. One `step(traverser)` does:

1. compute both players’ reaches top-down under the current strategy;
2. **price the net leaves** (the co-evolution): query V at every net leaf with the *current* normalized reaches, scale by opponent reach mass;
3. back values up (accumulate regret at traverser nodes, sum at opponent nodes, $\frac{1}{44}$ -average at chance nodes);
4. regret-matching $\rightarrow$ new current strategy; reach-weight it into the running average ($= \bar{\sigma}$, the converging object).

Listing 3: `subgame_solver.py` — the leaf-value scaling trick, the single most important detail. The net outputs values assuming the opponent belief is normalized to sum 1; the solver recovers the true counterfactual leaf value by multiplying by the opponent’s reach *mass* at that leaf. This decoupling lets one net generalize across subgames with wildly different reach magnitudes.

```
def _compute_net_leaf_values(self, traverser):
    opp = 1 - traverser
    queries = np.stack([encode_query(traverser, nd.public_state,
                                     self.reach[0][nid], self.reach[1][nid])
                       for nid in self.net_leaves])          # normalized inside
    vals = self.value_fn(queries)                             # (N, 1326), normalized
    for row, nid in enumerate(self.net_leaves):
        mass = self.reach[opp][nid].sum()                    # opponent reach MASS
        self.values[nid] = vals[row] * self.net_leaf_valid[nid] * mass
```

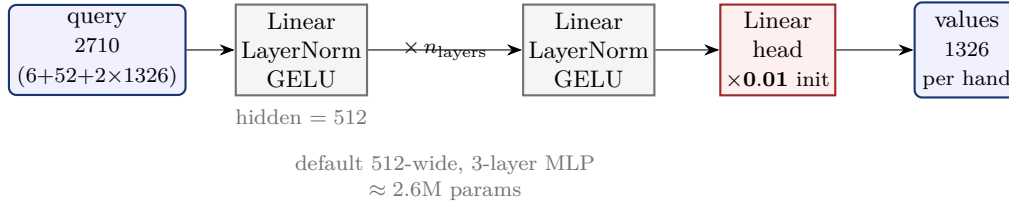
Watch out

The training target uses the *same* convention. `get_hand_values` returns `root_values_means` — the linear-CFR running mean of the root counterfactual values — computed with the root beliefs normalized to sum 1 (mass = 1). So “net output $\times$ opponent mass” at a leaf and “root value at mass 1” as a label are consistent. Get this wrong and the net learns a mis-scaled value function that silently caps how low exploitability can go.

The chance node couples turn and river inside one CFR. Reaching down, it masks each player’s reach by the dealt card; backing up, it averages the 48 river values with the matchup normalizer $\frac{1}{44}$ ($= 52 - 4 \text{ board} - 2 \text{ hero} - 2 \text{ opp}$). Best response (`exploitability.py`) handles chance the same way; its value at the root is the *exploitability gate* used everywhere.

5 The value network (`models.py`)

ReBeL’s value net is the AlphaZero analogue: $V(\text{PBS}) \rightarrow$ per-hand values. We follow the reference `Net2` exactly — a GELU MLP whose **output layer is scaled by 0.01 at init** so the net’s first predictions are near zero (a near-zero value function is the safe starting point for the bootstrapping below). One net spans all streets; the board and street live in the query.



Listing 4: `models.py` — the net and the numpy adapter the solver calls. `NetValueFn` is the bridge: numpy queries in, numpy values out, eval/no-grad, on the net’s device. Card-removal/opp-mass scaling is the solver’s job (previous section).

```
class ValueNet(nn.Module):
    def __init__(self, query_dim, n_hidden=512, n_layers=3, out_size=1326):
        self.body = build_mlp(n_in=query_dim, n_hidden=n_hidden,
                               n_layers=n_layers, use_layer_norm=True) # GELU
        self.head = nn.Linear(n_hidden, out_size)
        with torch.no_grad():
            self.head.weight *= 0.01; self.head.bias *= 0.01 # near-zero V

class NetValueFn: # queries (N,2710) -> values (N,1326)
    def __call__(self, queries):
        with torch.no_grad():
            return self.net(torch.as_tensor(queries, ...)).cpu().numpy()
```

Intuition

What the net sees and says (tutorial §“what the value net actually sees and says”). *Sees*: the PBS query — public scalars, board, both belief vectors. *Says*: for the queried traverser, the value of *each* of its 1326 hands at this PBS under near-equilibrium play. It is a *range*-valued function, which is why ReBeL “solves for your whole range, not your hand”: the solver needs values for every hand to run CFR, and one query returns all of them.

6 The self-play loop (`recursive.py`)

`RLRunner.step` is the tutorial’s six-step episode (`rebel_episode`), one hand of self-play. The correspondence is line-for-line:

tutorial step	<code>recursive.py</code>
(1) freeze PBS β_t	<code>current node_id + beliefs</code>
(2) build depth-limited subgame	<code>subtree_subgame(full, node_id, ., stop_at_chance)</code>
(3) solve with CFR (leaves co-evolve)	<code>CfrSolver(...).step × num_iters</code>
(4) store value + policy targets	<code>_emit: (query, root_values) per traverser</code>
(5) play a <i>random</i> CFR iterate	<code>run act_iter ~ U[0, K] steps, then sample</code>
(6) act + Bayes-update both beliefs	<code>_sample_next (decision) / _sample_chance (river card)</code>

Listing 5: `recursive.py` — one self-play hand. The “play a random iterate” safety trick (step 5) is the `act_iter` split: solve a uniformly-random number of CFR iterations, sample the next state from *that* iterate’s strategy, then finish solving and emit the average-strategy value as the label.

```
def step(self):
    node_id, beliefs = 0, uniform_beliefs(self.full.board)
    while not self.full.nodes[node_id].is_terminal:
        if self.full.nodes[node_id].is_chance: # river card: nature
            node_id, beliefs = self._sample_chance(node_id, beliefs); continue
```

```

solver = self._solve(node_id, beliefs)                # (2) build + (3) ...
act_iter = self.rng.integers(0, K + 1)                # (5) a RANDOM iterate
for i in range(act_iter): solver.step(i % 2)
next_id, next_beliefs = self._sample_next(node_id, solver, beliefs)
for i in range(act_iter, K): solver.step(i % 2)      # finish solving
self._emit(node_id, solver, beliefs)                 # (4) value targets
node_id, beliefs = next_id, next_beliefs             # (6) move on

```

Intuition

Why a random iterate? (tutorial Q2.) Two reasons baked into one line. *Training-distribution consistency*: the value net is queried at leaves with the *iterates*' beliefs, not just the equilibrium beliefs, so self-play must visit that same spread of beliefs — sampling the next state from a random iterate generates it. *Safety by randomization*: ReBeL's theorem says playing a uniformly-sampled CFR iterate is unexploitable in expectation, which a fixed average is not. Both come free from `act_iter`.

Cross-street. When the walk reaches the chance node, `_sample_chance` deals a river card (uniform over the unseen 48), removes it from both ranges, and descends to that river PBS, where the next loop turn solves the river *exactly* (last street, no net leaves). So one net is trained on *both* turn PBSs (bootstrapped from river net leaves) and river PBSs (exact targets) — the river anchors the bootstrap, as in the paper.

The gate, assembled consistently. `recursive_strategy` extracts a full-tree strategy for exploitability by solving each subgame *once* and reading interior strategies off that single solve (the reference `compute_strategy_recursive_to_leaf`), re-solving only at the depth-limit/chance leaves. Re-solving each node independently is *unsafe* resolving and leaves the strategy ~ 0.1 bb/hand exploitable even with a perfect value net — guarded by `test_recursive_assembly_exact`.

7 Training and the gate (`train.py`, `replay.py`, `exploitability.py`)

The single-process loop (`train/train_turn`) is: actors play self-play into a reservoir of (`query`, `value`) pairs; the net is regressed onto them (Huber loss); every few epochs the gate runs.

Listing 6: `train.py` — the loop and its gate. The gate measures the exploitability of the *recursively-resolved* strategy in the exact ground-truth tree; it must fall toward the exact-solve floor as the net learns.

```

for epoch in range(1, epochs+1):
    for _ in range(games_per_epoch): RlRunner(tree, vfn, params, buf, rng).step()
    for _ in range(batches_per_epoch):
        q, v = buf.sample(batch_size)
        loss = huber(net(q), v); loss.backward(); opt.step()      # value regression
    if epoch % eval_every == 0:
        e = exploitability_bb(tree, recursive_strategy(tree, vfn, p), uniform_beliefs)

```

Watch out

The value target carries the CFR bias. The label is the time-averaged root value, whose $O(1/T)$ averaging bias the net faithfully learns. Too few CFR iterations per solve (`cfr_iters`) and *every* label is biased, capping the trained net's exploitability regardless of net size or epochs — the default is 256, not 48. This was diagnosed by isolating the chain: exact recursion reaches 0.0003; the finite-iter value estimate sits at 0.04.

`replay.py` is an Algorithm-R reservoir over fixed-length (`query`, `value`) rows (unbiased uniform sample of the whole self-play stream). `exploitability.py` is best response over the public tree —

the correctness currency of the whole project.

8 Scaling the chance node (`showdown.py`)

The turn chance node has 48 river subtrees; naively each showdown/fold terminal is a dense 1326×1326 matrix-vector product (so ~ 1.7 GB of matrices per tree, the cause of multiprocess OOMs, and an $O(N^2)$ matvec per leaf per iteration). Both collapse to rank-vector cumulative sums.

Showdown (`RiverShowdown`). $v[h] = \text{stake} \cdot (\text{WIN}[h] - \text{LOSE}[h])$, with card removal by inclusion-exclusion over h 's two cards:

$$\text{WIN}[h] = \underbrace{\text{weaker_all}[h]}_{\text{rank-sorted cumsum}} - \text{weaker}_{c_0}[h] - \text{weaker}_{c_1}[h].$$

No “shares both cards” term: the only hand with both of h 's cards is h itself, never an opponent. **Fold** (`FoldRemoval`) is the same idea with no ranks: $\Sigma_{h' \cap h = \emptyset} = \text{total} - S[c_0] - S[c_1] + \text{reach}[h]$, $S[c] = \text{opponent mass on hands containing card } c$, via a *board-independent* incidence matrix.

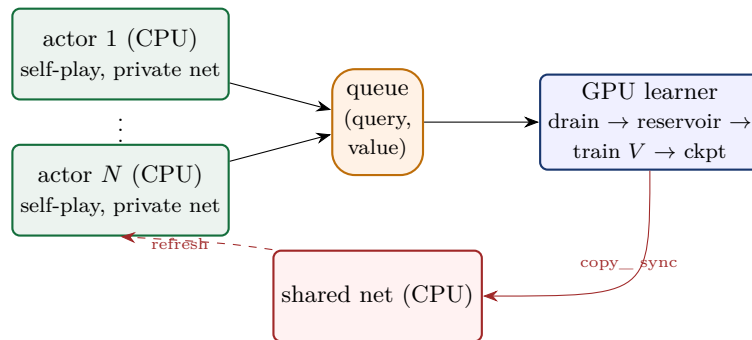
	dense matrices	rank/incidence	factor
per-card showdown	2.9 ms / 7 MB	0.13 ms / 0.1 MB	$22\times$ / $60\times$
turn-tree terminal memory	~ 1.7 GB	~ 5 MB	$\sim 340\times$
full turn $\rightarrow$ river solve	110 s	11 s	$10\times$
exploitability	0.02741	0.02741	identical

Watch out

A primitive replacing a dense terminal must reproduce its behaviour on *invalid* hands too. The dense *showdown* matrix zeroes invalid-hero rows, but the dense *fold* does not — and these values flow through best response's per-action max. Masking invalid hero on the fold silently shifted the measured exploitability $0.027 \rightarrow 0.0009$ (a falsely-low gate). The fix: `RiverShowdown` masks hero, `FoldRemoval` must not. Both validated exact (10^{-11}) for *every* hand.

9 The cluster trainer (`rebel_mp_gpu.py`)

Scaling the self-play to a node: N CPU actor processes run `RLRunner` with a shared-memory copy of the value net; a GPU learner drains a queue of (`query`, `value`) lists into a reservoir, trains, and syncs the shared net back. It mirrors the project's `cfr_mp_gpu` (forkserver bootstrap, `copy_weight_sync`).



Three bring-up gotchas, each a real bug and each documented in code: actors forward through a **private** net refreshed only *between* games (never read a tensor the learner is mid-`copy_weight_sync`); **BLAS**

pinned to one thread before NumPy loads (a multithreaded OpenBLAS pool does not survive process spawn); and a **watchdog respawns** dead actors. Checkpoints save the state dict plus a TorchScript trace for a future C++/FFI consumer.

10 Correspondence at a glance, and what is validated

ReBeL idea (tutorial)	file	key symbol
public belief state	<code>pbs.py</code>	<code>PublicState</code> , <code>encode_query</code>
depth-limited subgame (shape only)	<code>public_tree.py</code>	<code>build_*_subgame</code> , <code>subtree_subgame</code>
chance node / Bayes over board cards	<code>public_tree.py</code>	<code>Node.is_chance</code> , <code>_sample_chance</code>
CFR inside the subgame, leaves co-evolve	<code>subgame_solver.py</code>	<code>CfrSolver.step</code>
leaf = net output $\times$ opp mass	<code>subgame_solver.py</code>	<code>_compute_net_leaf_values</code>
value net $V(\text{PBS})$	<code>models.py</code>	<code>ValueNet</code> , <code>NetValueFn</code>
self-play episode (6 steps)	<code>recursive.py</code>	<code>RlRunner.step</code>
play a random CFR iterate (safety)	<code>recursive.py</code>	<code>act_iter</code> split
value target = root values of $\bar{\sigma}$	<code>subgame_solver.py</code>	<code>get_hand_values</code>
exploitability (the gate)	<code>exploitability.py</code>	<code>exploitability_bb</code>
training loop / reservoir	<code>train.py</code> , <code>replay.py</code>	<code>train</code> , <code>QueryValueBuffer</code>
scale (rank/incidence leaves)	<code>showdown.py</code>	<code>RiverShowdown</code> , <code>FoldRemoval</code>
cluster (actors + learner)	<code>rebel_mp_gpu.py</code>	<code>_actor_loop</code> , <code>main</code>

Validation (every layer has a hard gate).

- **Solver vs ground truth:** river endgames solve to $\sim 2 \times 10^{-4}$ bb/hand; 10 bb push/fold reproduces the Nash oracle (SB-jam/BB-call ranges agree $\sim 99\%$, ~ 3.6 mbb/hand exploitable).
- **Chance node:** full turn $\rightarrow$ river ground truth solves to 0.027 bb/hand; the all-in runout equity matches an independent `evaluate7` tally over the 44 valid river cards.
- **Value-net mechanism:** self-play drives the recursive strategy's exploitability $2.0 \rightarrow 0.04$ (river) and $0.25 \rightarrow 0.03$ (turn $\rightarrow$ river).
- **Scaling:** rank/incidence leaves match the dense matrices to 10^{-11} for all hands; exploitability byte-identical.

The code is the source of truth; this document is a reading guide. Where a claim here and the code disagree, the code (and its tests) win.